

segment customers based on their coffee preferences. By generating synthetic data, we can create a controlled environment for showcasing the clustering process. Synthetic data allows for reproducibility and facilitates learning by providing a simplified yet realistic representation of the problem domain. Our goal is to showcase how businesses can utilise clustering techniques to gain insights into customer segmentation, enabling them to strategise marketing campaigns and product offerings effectively.

We use the NumPy library to generate synthetic data simulating customer preferences for two coffee types: cappuccino and filter coffee. This synthetic data allows us to demonstrate the clustering process without relying on real-world data, making it accessible for educational purposes.

```
# importing required python libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

We utilise `np.random.seed(123)` to ensure reproducibility in our experiments. By setting the seed value to 123, we ensure that the same random numbers are generated each time we run the code. This is crucial for consistency in our results, especially when working with random processes. `n_samples = 1000` specifies the number of samples we will generate for our analysis, ensuring we have a sufficiently large dataset for meaningful insights.

```
# setting seed for reproducibility
np.random.seed(123)
n_samples = 1000
```

We generate random preferences for each customer by sampling from normal distributions. For male customers, we generate preferences for filter coffee with a mean of 0.61 and a standard deviation of 0.05, and preferences for cappuccino with a mean of 0.47 and a

standard deviation of 0.05. For female customers, we generate preferences for filter coffee with a mean of 0.4 and a standard deviation of 0.05, and preferences for cappuccino with a mean of 0.65 and a standard deviation of 0.05. These preferences are randomly generated to simulate diverse customer preferences in our dataset.

```
# Generate random preferences for each customer
male_filter_coffee = np.random.
normal(0.61, 0.05, size=(n_samples //
2,))
male_cappuccino = np.random.normal(0.47,
0.05, size=(n_samples // 2,))
female_cappuccino = np.random.
normal(0.65, 0.05, size=(n_samples //
2,))
female_filter_coffee = np.random.
normal(0.4, 0.05, size=(n_samples //
2,))
```

We combine the preferences of male and female customers for cappuccino and filter coffee into two separate arrays. By using the `np.concatenate()` function, we merge the arrays containing cappuccino preferences for males and females into a single array named `cappuccino_preferences`. Similarly, we merge the arrays containing filter coffee preferences for males and females into another array named `filter_coffee_preferences`. This consolidation allows us to analyse the combined preferences of both genders efficiently.

```
# Combine preferences of males and females
cappuccino_preferences =
np.concatenate((male_cappuccino, female_
cappuccino))
filter_coffee_preferences =
np.concatenate((male_filter_coffee,
female_filter_coffee))
```

We create a combined dataset called `preferences`, where each row represents a customer's preference for cappuccino and filter coffee. By using

the `np.column_stack()` function, we stack the arrays containing cappuccino and filter coffee preferences side by side, forming a 2D array. Additionally, we create an array called `gender_labels` to denote the gender of each customer. We assign 'Male' labels to the first half of the samples and 'Female' labels to the second half, ensuring that each set of preferences is associated with its corresponding gender. This combined dataset allows us to analyse and visualise the preferences of both male and female customers effectively.

```
# Combine preferences and gender labels
preferences = np.column_
stack((cappuccino_preferences, filter_
coffee_preferences))
gender_labels = np.array(['Male'] * (n_
samples // 2) + ['Female'] * (n_samples
// 2))
```

We perform K-means clustering to identify distinct clusters within the customer preference data. By specifying the number of clusters as 2 and setting a random state for reproducibility, we initialise the K-means object. Next, we fit the K-means model to the customer preferences data stored in the 'preferences' array. After fitting the model, we obtain the coordinates of the cluster centres using the 'cluster_centers_' attribute. Additionally, we obtain the cluster labels assigned to each data point using the 'labels_' attribute.

```
# Perform K-means clustering
kmeans = KMeans(n_clusters=2, random_
state=42)
kmeans.fit(preferences)
cluster_centers = kmeans.cluster_
centers_
cluster_labels = kmeans.labels_
```

We create a visualisation of the clusters using Matplotlib. By calling `plt.figure(figsize=(10, 6))`, we specify